Итоговый проект: улучшение EcomAds на основе данных

Учебный A/B-тест на синтетических данных

Исследуем, помогает ли заметная кнопка «Посмотреть рекомендации» чаще открывать рекомендации. Полный цикл: бизнес-задача → гипотеза → дизайн эксперимента → генерация и проверка данных → статистический анализ → рекомендации.

Важно: эксперимент смоделирован, а не проведён на реальных клиентах. Все вероятности и объёмы — учебные допущения. Результат демонстрирует метод анализа и не доказывает эффективность кнопки в реальном EcomAds.

Запуск: Python 3.10+, установить зависимости из `requirements.txt`, затем выполнить **Restart Kernel and Run All**. Ноутбук уже содержит результаты выполнения и графики. Интернет и внешние файлы для расчётов не нужны.

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import scipy
from scipy import stats
import matplotlib
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

SEED = 42
ALPHA = 0.05
POWER = 0.80
plt.rcParams.update({"figure.figsize": (8, 4), "font.size": 11, "axes.spines.top": False,
                     "axes.spines.right": False, "font.family": "DejaVu Sans"})
print(f"Python {sys.version.split()[0]} | NumPy {np.__version__} | pandas {pd.__version__}")
print(f"SciPy {scipy.__version__} | Matplotlib {matplotlib.__version__} | seed = {SEED}")
```

```
Python 3.12.1 | NumPy 2.2.0 | pandas 2.2.3
SciPy 1.14.1 | Matplotlib 3.9.4 | seed = 42
```

1. Бизнес-контекст

1.1. Описание продукта

EcomAds — веб-сервис анализа рекламы на Wildberries. Он показывает рекламные показатели, находит отклонения и предлагает рекомендации по кампаниям и ключевым словам. Описание опирается на документацию MVP в репозитории: `docs/architecture/ecomads-c4-homework.md`.

Целевая аудитория: продавцы Wildberries и менеджеры, которые управляют рекламой магазинов.

Проблема: пользователям сложно быстро понять, на какие показатели обратить внимание. Рекомендации помогают сориентироваться, но пользователь может не заметить переход к ним. Последнее — предположение для учебной гипотезы, а не установленный факт.

1.2. Бизнес-метрика

Конверсия в открытие хотя бы одной рекомендации за 24 часа после первого показа дашборда в эксперименте.

Конверсия = число аккаунтов, открывших рекомендацию / число всех распределённых аккаунтов. Один аккаунт даёт одно наблюдение: 1 — открыл, 0 — не открыл.

Метрика показывает, доходят ли пользователи до основной полезной функции сервиса. Это ранняя продуктовая метрика, а не показатель прибыли: само открытие ещё не означает применение рекомендации или экономию рекламного бюджета.

2. Формулировка гипотезы

2.1. Продуктовая гипотеза

Если добавить на дашборд заметную кнопку «Посмотреть рекомендации», то конверсия в открытие рекомендации за 24 часа вырастет на 30% относительно базового уровня, потому что переход станет проще найти.

Предполагаемый базовый уровень — 30%, целевой — 39%. Это **+9 процентных пунктов (п.п.)**, или **+30% относительно исходной конверсии**: (39 − 30) / 30. Оба значения заданы для учебного примера; исторических данных о базовой конверсии нет.

2.2. Статистические гипотезы

- **H_0:** p_A = p_B — конверсии одинаковы.
- **H_1:** p_A ≠ p_B — конверсии различаются.

Здесь p_A и p_B — вероятности открытия рекомендации в контроле и эксперименте. Используем двустороннюю проверку: кнопка может как помочь, так и ухудшить результат. Чтобы поддержать продуктовую гипотезу, значимое различие должно быть положительным. Тест проверяет отличие от нуля, а не доказывает достижение эффекта ровно +30%.

2.3. Параметры

- **α = 0,05:** допустимая вероятность ошибки первого рода при верной H_0 — 5%.
- **Мощность = 80%:** планируем обнаруживать эффект 30% → 39% примерно в 80% подобных экспериментов.
- **Минимальный обнаруживаемый эффект (MDE):** 9 п.п. Меньший эффект может остаться незамеченным.

Это стандартный компромисс между чувствительностью и объёмом учебного эксперимента. Порог α и единственная основная метрика фиксируются до генерации данных.

3. Дизайн эксперимента и данные

3.1. План A/B-теста

- **A, контроль:** текущий интерфейс дашборда.
- **B, эксперимент:** тот же интерфейс с заметной кнопкой «Посмотреть рекомендации».
- Включаем аккаунты продавцов с загруженной статистикой и хотя бы одной доступной рекомендацией.
- Единица рандомизации и анализа — аккаунт продавца. Все его сотрудники и повторные визиты видят одну версию.
- Распределяем аккаунты 1:1 случайно, назначение сохраняем. Каждый аккаунт включаем один раз.
- В анализ входят все распределённые аккаунты, в том числе не открывшие рекомендации (принцип intention-to-treat).

- Для каждого аккаунта ждём полные 24 часа; отсутствие события после полного окна означает 0 только при исправном трекинге.
- Учебный график: дни 1–6 — набор, день 7 — завершение последних окон и анализ. Досрочно по p-value не останавливаемся.

Для реального MVP срок зависит от трафика: 1 000 подходящих аккаунтов за неделю — учебное допущение, а не прогноз. Если трафика мало, нужно заранее увеличить срок набора.

Расчёт выборки. Используем нормальное приближение для сравнения двух независимых долей, равные группы, двусторонний тест. В формуле $\bar{p} = (p_A + p_B) / 2$:

$$n = \frac{[z_{1-\alpha/2}\sqrt{2\bar{p}(1-\bar{p})} + z_{power}\sqrt{p_A(1-p_A)+p_B(1-p_B)}]^2}{(p_B - p_A)^2}.$$

```
p_A_plan, p_B_plan = 0.30, 0.39
p_bar = (p_A_plan + p_B_plan) / 2
z_alpha = stats.norm.ppf(1 - ALPHA / 2)
z_power = stats.norm.ppf(POWER)
n_raw = ((z_alpha * np.sqrt(2 * p_bar * (1 - p_bar))
          + z_power * np.sqrt(p_A_plan * (1 - p_A_plan)
                              + p_B_plan * (1 - p_B_plan))) ** 2
         / (p_B_plan - p_A_plan) ** 2)
n_required = int(np.ceil(n_raw))
N = 500  # округляем вверх до удобного размера каждой группы
assert N >= n_required
print(f"Нужно минимум {n_required} аккаунтов в каждой группе.")
print(f"План: A = {N}, B = {N}; всего {2 * N} аккаунтов.")
```

```
Нужно минимум 437 аккаунтов в каждой группе.
План: A = 500, B = 500; всего 1000 аккаунтов.
```

3.2. Генерация синтетических данных

Генерируем по 500 независимых исходов Бернулли в каждой группе с вероятностями 0,30 и 0,39. Затем случайно сопоставляем группы уникальным аккаунтам при фиксированных размерах групп. Вероятность успеха — параметр симуляции; наблюдаемая доля будет случайно отличаться от него. Seed = 42 обеспечивает воспроизводимость; seed и данные не подбираются по значимости.

Схема: `account_id` — уникальный аккаунт, `group` — A/B, `opened_recommendation` — событие 0/1 за полное 24-часовое окно. В этой упрощённой модели нет потерь событий и незавершённых окон.

```
rng = np.random.default_rng(SEED)
opened_A = rng.binomial(n=1, p=p_A_plan, size=N)
opened_B = rng.binomial(n=1, p=p_B_plan, size=N)
df = pd.DataFrame({
    "group": np.repeat(["A", "B"], N),
    "opened_recommendation": np.concatenate([opened_A, opened_B]),
})
df = df.iloc[rng.permutation(len(df))].reset_index(drop=True)
df.insert(0, "account_id", np.arange(1, len(df) + 1))
display(df.head(10))
print("Размер таблицы:", df.shape)
```

	account_id	group	opened_recommendation
0	1	B	1
1	2	B	1
2	3	B	1
3	4	B	1
4	5	A	0
5	6	B	0
6	7	A	0
7	8	A	0
8	9	B	0
9	10	A	1

```
Размер таблицы: (1000, 3)
```

3.3. Проверка качества данных

Проверяем пропуски, дубли аккаунтов, допустимые группы и значения целевой метрики. Для бинарной метрики выброс — значение вне {0, 1}; корректные нули и единицы удалять нельзя. В реальном эксперименте дополнительно проверяли бы полноту событий и созревание 24-часовых окон.

```
quality = pd.Series({
    "Пропущенные значения": int(df.isna().sum().sum()),
    "Дубли account_id": int(df.account_id.duplicated().sum()),
    "Неизвестные группы": int((~df.group.isin(["A", "B"])).sum()),
    "Значения метрики вне 0/1": int((~df.opened_recommendation.isin([0, 1])).sum()),
}, name="Число ошибок")
display(quality.to_frame())
assert (quality == 0).all(), "Найдены ошибки качества данных"
assert df.groupby("group").size().to_dict() == {"A": N, "B": N}
summary = df.groupby("group").agg(
    accounts=("account_id", "size"),
    opened=("opened_recommendation", "sum"),
    conversion=("opened_recommendation", "mean"),
    std=("opened_recommendation", "std"),
    minimum=("opened_recommendation", "min"),
    maximum=("opened_recommendation", "max"),
)
summary["not_opened"] = summary.accounts - summary.opened
display(summary.round(4))
print("Ошибок нет. Удалять или заменять наблюдения не требуется.")
```

	Число ошибок
Пропущенные значения	0
Дубли account_id	0
Неизвестные группы	0
Значения метрики вне 0/1	0

group	accounts	opened	conversion	std	minimum	maximum	not_opened
A	500	145	0.290	0.4542	0	1	355
B	500	191	0.382	0.4864	0	1	309

```
Ошибок нет. Удалять или заменять наблюдения не требуется.
```

4. Статистический анализ

4.1. Проверка предпосылок

Нормальность: исходная метрика бинарная, поэтому нормально распределённой быть не может. Для выполнения пункта задания покажем Shapiro–Wilk. Его результат ожидаем, он не выбирает основной тест.

Однородность дисперсий: покажем тест Левена. У Бернулли дисперсия равна p(1 − p), поэтому при разных конверсиях дисперсии также могут различаться. Равенство дисперсий исходных нулей и единиц не является предпосылкой используемого теста долей.

Независимость: не устанавливается p-value. В симуляции обеспечена независимой генерацией и одной строкой на аккаунт. В реальности нужны постоянное назначение версии на уровне аккаунта и отсутствие влияния между аккаунтами. Общие менеджеры нескольких аккаунтов могут нарушить независимость; тогда нужна рандомизация на уровне менеджера/организации.

```
A = df.loc[df.group == "A", "opened_recommendation"].to_numpy()
B = df.loc[df.group == "B", "opened_recommendation"].to_numpy()
assumptions = []
for label, values in [("Shapiro–Wilk, A", A), ("Shapiro–Wilk, B", B)]:
    result = stats.shapiro(values)
    assumptions.append({"Проверка": label, "Статистика": result.statistic, "p-value": result.pvalue})
levene = stats.levene(A, B, center="mean")
assumptions.append({"Проверка": "Левен: равенство дисперсий", "Статистика": levene.statistic,
                    "p-value": levene.pvalue})
display(pd.DataFrame(assumptions).style.format({"Статистика": "{:.4f}", "p-value": "{:.3g}"}))
print("Нормальность отвергается; для бинарной метрики это ожидаемо.")
print("По Левену:", "равенство дисперсий отвергается." if levene.pvalue < ALPHA
      else "нет оснований отвергать равенство дисперсий.")
print("Повторных аккаунтов:", df.account_id.duplicated().sum())
display(summary[["opened", "not_opened"]])
assert summary[["opened", "not_opened"]].to_numpy().min() >= 10
print("В обеих группах минимум 10 успехов и 10 неуспехов: нормальное приближение для долей допустимо.")
```

	Проверка	Статистика	p-value
0	Shapiro–Wilk, A	0.5690	2.77e-33
1	Shapiro–Wilk, B	0.6160	7.54e-32
2	Левен: равенство дисперсий	36.7408	1.91e-09

```
Нормальность отвергается; для бинарной метрики это ожидаемо.
По Левену: равенство дисперсий отвергается.
Повторных аккаунтов: 0
```

group	opened	not_opened
A	145	355
B	191	309

```
В обеих группах минимум 10 успехов и 10 неуспехов: нормальное приближение для долей допустимо.
```

4.2. Выбор и применение теста

Используем **двусторонний z-тест для двух независимых долей**: сравниваем бинарный исход между двумя независимыми группами с достаточным числом успехов и неуспехов. Нормальность самих нулей и единиц не нужна: используется приближение распределения разности долей.

При H_0 общая оценка конверсии p_pool = (x_A + x_B) / (n_A + n_B):

$$z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}_{pool}(1-\hat{p}_{pool})(1/n_A + 1/n_B)}}.$$

Для разности долей строим приближённый 95%-й интервал Вальда с **необъединённой** стандартной ошибкой. Для каждой конверсии на графике используем интервал Уилсона. Интервал разности помогает оценить величину и неопределённость эффекта, а p-value — проверить H_0.

```python
n_A, n_B = len(A), len(B)
x_A, x_B = int(A.sum()), int(B.sum())
p_A, p_B = A.mean(), B.mean()
delta = p_B - p_A
relative_uplift = delta / p_A
p_pool = (x_A + x_B) / (n_A + n_B)
se_null = np.sqrt(p_pool * (1 - p_pool) * (1 / n_A + 1 / n_B))
z_stat = delta / se_null
p_value = 2 * stats.norm.sf(abs(z_stat))
se_delta = np.sqrt(p_A * (1 - p_A) / n_A + p_B * (1 - p_B) / n_B)
ci_low, ci_high = delta - z_alpha * se_delta, delta + z_alpha * se_delta
reject_null = p_value < ALPHA

print(f"A: {x_A}/{n_A} = {p_A:.1%}")
print(f"B: {x_B}/{n_B} = {p_B:.1%}")
print(f"Абсолютный эффект B − A: {delta * 100:.2f} п.п.")
print(f"Относительный прирост: {relative_uplift:.1%}")
print(f"z = {z_stat:.4f}; p-value = {p_value:.6f}")
print(f"95%-й ДИ разности: [{ci_low * 100:.2f}; {ci_high * 100:.2f}] п.п.")
print("Решение:", "отвергаем H₀" if reject_null else "не отвергаем H₀")
```

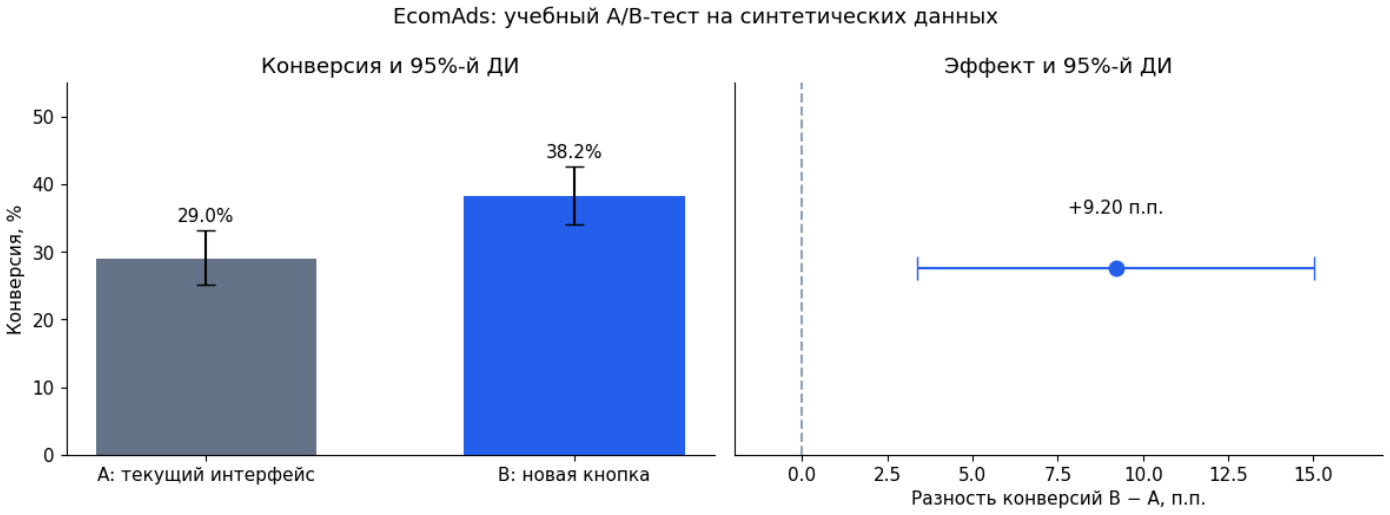
```
A: 145/500 = 29.0%
B: 191/500 = 38.2%
Абсолютный эффект B − A: 9.20 п.п.
Относительный прирост: 31.7%
z = 3.0797; p-value = 0.002072
95%-й ДИ разности: [3.37; 15.03] п.п.
Решение: отвергаем H₀
```

4.3. Визуализация результатов

На первом графике — конверсии с 95%-ми интервалами Уилсона. На втором — эффект B − A с 95%-м интервалом. Пунктир на втором графике показывает отсутствие эффекта. Решение о значимости принимается по тесту разности, а не по тому, пересекаются ли интервалы отдельных конверсий.

```
def wilson_interval(successes, total, z=z_alpha):
    p = successes / total
    denominator = 1 + z**2 / total
    center = (p + z**2 / (2 * total)) / denominator
    half = z * np.sqrt(p * (1 - p) / total + z**2 / (4 * total**2)) / denominator
    return center - half, center + half

rates = np.array([p_A, p_B])
intervals = np.array([wilson_interval(x_A, n_A), wilson_interval(x_B, n_B)])
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].bar(["A: текущий интерфейс", "B: новая кнопка"], rates * 100,
            color=["#64748b", "#2563eb"], width=0.6)
axes[0].errorbar([0, 1], rates * 100,
                 yerr=np.vstack([rates - intervals[:, 0], intervals[:, 1] - rates]) * 100,
                 fmt="none", color="black", capsize=6)
for i, rate in enumerate(rates):
    axes[0].text(i, intervals[i, 1] * 100 + 1.4, f"{rate:.1%}", ha="center")
axes[0].set(ylim=(0, 55), ylabel="Конверсия, %", title="Конверсия и 95%-й ДИ")
axes[1].errorbar(delta * 100, 0, xerr=z_alpha * se_delta * 100,
                 fmt="o", color="#2563eb", capsize=7, markersize=9)
axes[1].axvline(0, color="#94a3b8", linestyle="--")
axes[1].set(yticks=[], xlabel="Разность конверсий B − A, п.п.", title="Эффект и 95%-й ДИ")
axes[1].set_xlim(min(-2, ci_low * 100 - 2), max(2, ci_high * 100 + 2))
axes[1].text(delta * 100, 0.09, f"{delta * 100:+.2f} п.п.", ha="center")
axes[1].set_ylim(-0.3, 0.3)
fig.suptitle("EcomAds: учебный A/B-тест на синтетических данных", fontsize=13)
fig.tight_layout()
plt.show()
```



5. Интерпретация и выводы

5.1. Статистический вывод

Следующая ячейка подставляет фактические результаты расчёта, поэтому вывод не расходится с данными. P-value — вероятность при верной H_0 получить наблюдаемое или более экстремальное различие. Это не вероятность истинности H_0 и не вероятность того, что кнопка полезна.

```
decision = "отвергаем H₀" if reject_null else "не отвергаем H₀"
interpretation = (
    "В смоделированном эксперименте обнаружено статистически значимое увеличение конверсии."
    if reject_null and delta > 0 else
    "В смоделированном эксперименте обнаружено статистически значимое снижение конверсии."
    if reject_null else
    "Данных недостаточно, чтобы сделать вывод о различии конверсий; это не доказывает отсутствие эффекта."
)
display(Markdown(f"""
**p-value = {p_value:.6f}; α = {ALPHA}. Мы {decision}.**

{interpretation}

В A конверсия составила **{p_A:.1%}**, в B — **{p_B:.1%}**.
Оценка эффекта: **{delta * 100:+.2f} п.п.**, или **{relative_uplift:+.1%} относительно контроля**.
95%-й доверительный интервал абсолютного эффекта: **от {ci_low * 100:.2f} до {ci_high * 100:.2f} п.п.**

Запланированные +9 п.п. использовались для расчёта выборки. Наблюдаемая точечная оценка
не доказывает, что истинный эффект достигает этого порога.
"""))
```

p-value = 0.002072; α = 0.05. Мы отвергаем H_0.

В смоделированном эксперименте обнаружено статистически значимое увеличение конверсии.

В A конверсия составила **29.0%**, в B — **38.2%**. Оценка эффекта: **+9.20 п.п.**, или **+31.7% относительно контроля**. 95%-й доверительный интервал абсолютного эффекта: **от 3.37 до 15.03 п.п.**

Запланированные +9 п.п. использовались для расчёта выборки. Наблюдаемая точечная оценка не доказывает, что истинный эффект достигает этого порога.

5.2. Бизнес-вывод и практическая значимость

Заметная кнопка потенциально помогает большему числу аккаунтов дойти до рекомендаций. Для небольшой интерфейсной доработки рост на несколько процентных пунктов может быть полезен, но окончательное решение зависит от стоимости разработки и того, применяют ли пользователи рекомендации. Пересчитаем эффект в число дополнительных аккаунтов на 1 000 посетивших дашборд. Это сценарный расчёт при переносимости эффекта, а не прогноз выручки.

```
display(Markdown(f"""
При таком же эффекте на каждые **1 000 подходящих аккаунтов** получили бы примерно
**{delta * 1000:.0f} дополнительных аккаунтов**, открывших рекомендацию.
Интервал неопределённости: примерно **от {ci_low * 1000:.0f} до {ci_high * 1000:.0f}**.

Статистическая значимость сама по себе не гарантирует окупаемость. По этой метрике
нельзя утверждать, что снизились рекламные расходы или выросли продажи.
"""))
```

При таком же эффекте на каждые **1 000 подходящих аккаунтов** получили бы примерно **92 дополнительных аккаунтов**, открывших рекомендацию. Интервал неопределённости: примерно **от 34 до 150**.

Статистическая значимость сама по себе не гарантирует окупаемость. По этой метрике нельзя утверждать, что снизились рекламные расходы или выросли продажи.

5.3. Рекомендации

Решение в рамках учебного кейса: рекомендуем внедрить заметную кнопку «Посмотреть рекомендации». В эксперименте конверсия выросла с 29,0% до 38,2%: на 9,2 п.п., или на 31,7% относительно контроля. Различие статистически значимо (p-value = 0,0021), а 95%-й интервал эффекта — от +3,37 до +15,03 п.п. — не включает ноль. Результат поддерживает гипотезу о положительном влиянии кнопки на открытия рекомендаций. Точечная оценка выше целевых +9 п.п., но интервал не позволяет утверждать, что истинный эффект обязательно достигает этой цели.

1. **Сделать кнопку постоянным элементом дашборда.** Это конкретное изменение соответствует варианту B, который показал более высокую конверсию. Рекомендация предполагает небольшие затраты на доработку интерфейса; стоимость и окупаемость в эксперименте не измерялись.
2. **Внедрять постепенно и контролировать работу интерфейса:** ошибки открытия рекомендаций и скорость дашборда. Эти показатели не измерялись в учебном эксперименте, поэтому отсутствие технических проблем не доказано.
3. **После внедрения отслеживать конверсию в том же 24-часовом окне.** Проверять, сохраняется ли повышение после периода новизны. Такое наблюдение полезно для мониторинга, но само по себе не заменяет причинную оценку эффекта.
4. **Следующий приоритет аналитики — применение рекомендаций.** Измерять долю пользователей, которые после открытия принимают рекомендацию, и их возврат в сервис. Рост открытий ещё не доказывает рост пользы или выручки.

Эти рекомендации относятся к результатам смоделированного эксперимента. Для реального EcomAds синтетические данные не являются достаточным основанием для выпуска; потребуется проверка на реальных пользователях.

Ограничения исследования

- Данные полностью синтетические; положительный эффект заложен в вероятности генерации. Значимый результат демонстрирует работу анализа, а не подтверждает реальную продуктовую гипотезу.
- Базовая конверсия 30% и MDE 9 п.п. — предположения, а не результаты исторических измерений.
- Модель не учитывает различия продавцов, сезонность, технические сбои и эффект новизны.
- Окно 24 часа оценивает только краткосрочное действие; выручка, retention и качество рекомендаций не измерялись.
- Вывод опирается на независимость аккаунтов; в реальности её нужно обеспечить дизайном.
- Нормальные приближения подходят для этой выборки; при редких событиях и малых группах нужен иной расчёт.

Итог учебного кейса: по результатам эксперимента рекомендуем внедрить кнопку и отслеживать дальнейшее использование рекомендаций. Ограничение переноса этого решения в реальный продукт — синтетическая природа данных.